

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Certificates ▼](#)[In](#)[HTML](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)[C](#)[C](#)

NumPy Tutorial

[NumPy HOME](#)[NumPy Intro](#)[NumPy Getting Started](#)[NumPy Creating Arrays](#)[NumPy Array Indexing](#)[NumPy Array Slicing](#)[NumPy Data Types](#)[NumPy Copy vs View](#)[NumPy Array Shape](#)[NumPy Array Reshape](#)[NumPy Array Iterating](#)[NumPy Array Join](#)[NumPy Array Split](#)[NumPy Array Search](#)[NumPy Array Sort](#)[NumPy Array Filter](#)

NumPy Random

[Random Intro](#)[Data Distribution](#)[Random Permutation](#)[Seaborn Module](#)[Normal Distribution](#)[Binomial Distribution](#)[Poisson Distribution](#)[Uniform Distribution](#)[Logistic Distribution](#)[Multinomial Distribution](#)[Exponential Distribution](#)[Chi Square Distribution](#)[Rayleigh Distribution](#)[Pareto Distribution](#)

NumPy Array Iterating

[< Previous](#)[Next >](#)

Iterating Arrays

Iterating means going through elements one by one.

As we deal with multi-dimensional arrays in numpy, we can do this using basic **for** loop of python.

If we iterate on a 1-D array it will go through each element one by one.

Example

[Get your own Python Server](#)

Iterate on the elements of the following 1-D array:

```
import numpy as np

arr = np.array([1, 2, 3])

for x in arr:
    print(x)
```

[Try it Yourself »](#)

ADVERTISEMENT

Iterating 2-D Arrays

In a 2-D array it will go through all the rows.

Example

Iterate on the elements of the following 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    print(x)
```

[Try it Yourself »](#)

If we iterate on a n -D array it will go through $n-1$ th dimension one by one.

To return the actual values, the scalars, we have to iterate the arrays in each dimension.

Example

Iterate on each scalar element of the 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    for y in x:
        print(y)
```

Get Certified

Document your skills
W3Schools Certificate

\$1,995

Save 65%

COLOR
PICKER



ADVERTISEMENT

[Try it Yourself »](#)

ADVERTISEMENT

[REMOVE ADS](#)

Iterating 3-D Arrays

In a 3-D array it will go through all the 2-D arrays.

Example

Iterate on the elements of the following 3-D array:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]],
               [[7, 8, 9], [10, 11, 12]]])

for x in arr:
    print(x)
```

[REMOVE
ADS](#)[Try it Yourself »](#)

To return the actual values, the scalars, we have to iterate the arrays in each dimension.

Example

Iterate down to the scalars:

```
import numpy as np
```

```
arr = np.array([[[1, 2, 3], [4, 5, 6]],  
               [[7, 8, 9], [10, 11, 12]]])  
  
for x in arr:  
    for y in x:  
        for z in y:  
            print(z)
```

Try it Yourself »

Iterating Arrays Using `nditer()`

The function `nditer()` is a helping function that can be used from very basic to very advanced iterations. It solves some basic issues which we face in iteration, lets go through it with examples.

Iterating on Each Scalar Element

In basic `for` loops, iterating through each scalar of an array we need to use n `for` loops which can be difficult to write for arrays with very high dimensionality.

Example

Iterate through the following 3-D array:

```
import numpy as np  
  
arr = np.array([[[1, 2], [3, 4]], [[5,  
6], [7, 8]]])  
  
for x in np.nditer(arr):  
    print(x)
```

[Try it Yourself »](#)

Iterating Array With Different Data Types

We can use `op_dtypes` argument and pass it the expected datatype to change the datatype of elements while iterating.

NumPy does not change the data type of the element in-place (where the element is in array) so it needs some other space to perform this action, that extra space is called buffer, and in order to enable it in `nditer()` we pass `flags=['buffered']`.

Example

Iterate through the array as a string:

```
import numpy as np

arr = np.array([1, 2, 3])

for x in np.nditer(arr, flags=
['buffered'], op_dtypes=['S']):
    print(x)
```

[Try it Yourself »](#)

Iterating With Different Step Size

We can use filtering and followed by iteration.

Example

Iterate through every scalar element of the 2D array skipping 1 element:

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

for x in np.nditer(arr[:, ::2]):
    print(x)
```

Try it Yourself »

Enumerated Iteration Using ndenumerate()

Enumeration means mentioning sequence number of somethings one by one.

Sometimes we require corresponding index of the element while iterating, the `ndenumerate()` method can be used for those usecases.

Example

Enumerate on following 1D arrays elements:

```
import numpy as np

arr = np.array([1, 2, 3])

for idx, x in np.ndenumerate(arr):
    print(idx, x)
```

Try it Yourself »

Example

Enumerate on following 2D array's elements:

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

for idx, x in np.ndenumerate(arr):
    print(idx, x)
```

Try it Yourself »

Exercise [?]

Consider the following code:

```
import numpy as np
arr = np.array(['a', 'b', 'c'])
for x in arr:
    print('Hello ' + x)
```

What will be the printed result?

- ☐ Hello a b c
- ☐ Hello ['a', 'b', 'c']
- ☐ Hello a
Hello b
Hello c

Submit Answer »